

PLANO DE QUALIDADE DE SOFTWARE

Rota Fácil - Planejamento e Arquitetura de Testes

A qualidade de software passa a ser tratada como uma propriedade incorporada ao ciclo de desenvolvimento, com validações definidas antes da execução e critérios objetivos para decidir quando uma alteração pode avançar. O planejamento parte da necessidade de substituir verificações ocasionais por um fluxo repetível de validação, no qual requisitos, regras de negócio, integrações e fluxos críticos tenham formas de verificação compatíveis com o risco que representam.

O Rota Fácil é especificado como um sistema de gestão e acompanhamento de rotas de transporte escolar, com interação entre Prefeitura, Motorista e Aluno. Para este planejamento, a arquitetura considerada corresponde à implementação atual, formada por um core backend estruturado como monólito modular, um serviço separado de IA destinado às análises, um frontend web e um aplicativo mobile. O RabbitMQ é utilizado especificamente como mecanismo de comunicação assíncrona entre o core e o serviço de IA. O PostgreSQL permanece como base de persistência do core e Docker/Docker Compose fornece a infraestrutura necessária para execução dos componentes. Nesse contexto, a estratégia de qualidade precisa validar principalmente as regras e módulos do core, as integrações com persistência e, de forma específica, o fluxo assíncrono que conecta o core ao serviço de IA, além das jornadas críticas realizadas pelas interfaces web e mobile.

Escopo que orienta a validação

O escopo inicial está diretamente ligado aos requisitos funcionais, aos requisitos não funcionais e às histórias de usuário especificadas para o sistema. A validação deverá priorizar os comportamentos que controlam acesso, alteram o estado de viagens, registram presença, propagam eventos ou mantêm a rastreabilidade das operações.

No domínio de acesso, serão protegidas a autenticação por login e senha e a diferenciação de permissões entre Prefeitura, Motorista e Aluno. Também entram no escopo o cadastro, a edição, a exclusão e o vínculo de usuários a uma prefeitura, respeitando as restrições definidas para cada perfil. Esses pontos são sustentados pelos requisitos RF001 a RF004 e pelos requisitos não funcionais de segurança e controle de acesso.

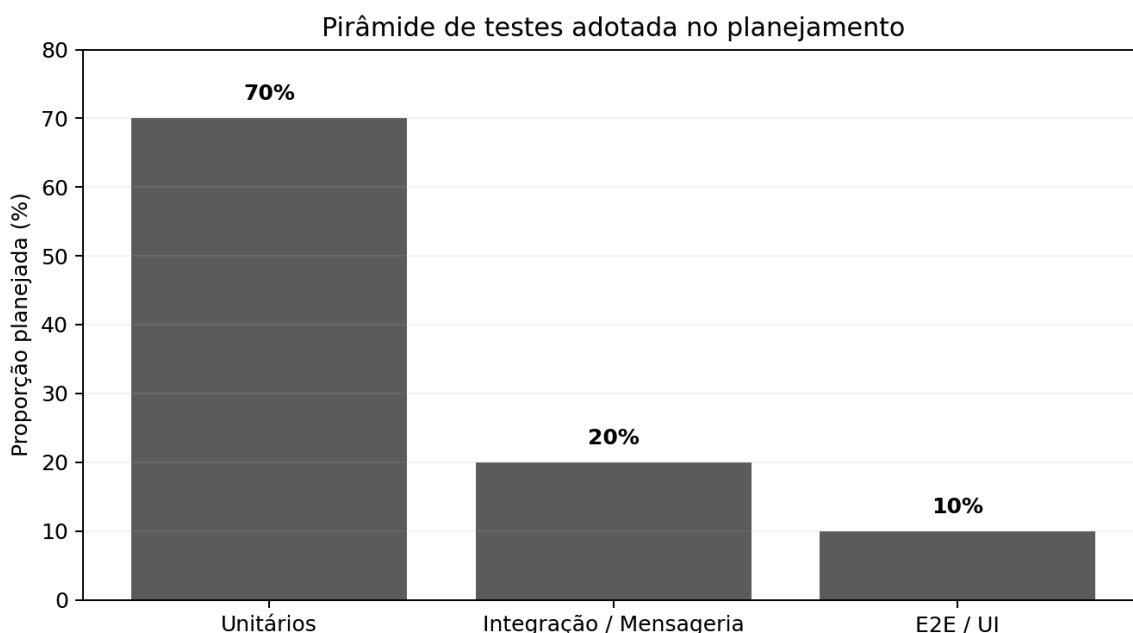
No domínio das viagens, a validação abrangerá a visualização de viagens, a inscrição e o cancelamento da participação de alunos, o início, a finalização e o cancelamento de viagens por motoristas, a visualização de alunos e paradas, a geração de QR Code e o registro de check-in. O comportamento do sistema também deverá ser verificado quando uma viagem é cancelada, considerando a notificação dos usuários vinculados, e quando uma viagem é finalizada, considerando o registro de ausência para alunos sem check-in.

No domínio administrativo, entram no escopo o gerenciamento de rotas, ônibus, motoristas e instituições, a geração de relatórios, o registro de auditoria e a visualização das informações autorizadas de perfil. A documentação também descreve notificações, processamento de eventos por mensageria e mecanismos

automatizados de registro de ausência; esses comportamentos serão tratados como integrações relevantes porque dependem da interação entre regras de negócio e serviços.

Uma distribuição de testes proporcional ao risco

A estrutura atual favorece uma concentração maior da validação em níveis próximos ao monólito modular, reservando a parcela intermediária para os pontos em que o core depende de infraestrutura externa ou se comunica com o serviço de IA. A composição planejada da suíte será de 70% de testes unitários, 20% de testes de integração e mensageria e 10% de testes de sistema ponta a ponta pelas interfaces web e mobile. Os percentuais representam uma meta de organização da suíte, não uma medição já realizada.



A camada unitária concentrará as regras que podem ser verificadas isoladamente. Isso inclui validações de domínio, transformação de dados, decisões condicionais e outras regras que não precisam de PostgreSQL, RabbitMQ ou interface para serem demonstradas. A maior participação dessa camada reduz o custo de execução e aproxima a falha do ponto em que o comportamento foi implementado.

A camada de integração cobrirá os casos em que o comportamento correto depende da persistência do core, de contratos de API ou da comunicação com o serviço de IA. No Rota Fácil, isso inclui a interação com PostgreSQL e os cenários em que RabbitMQ participa do processamento assíncrono entre o core e a IA. Nessa camada, não basta verificar apenas o caminho de sucesso: a publicação, o consumo, a resposta esperada do serviço de IA e o comportamento diante de falhas precisam ser observados de forma controlada.

A camada E2E permanecerá deliberadamente pequena. Ela será utilizada para validar os fluxos críticos que precisam ser demonstrados como uma jornada completa, incluindo as interações pela interface e a passagem pelos componentes necessários para concluir o comportamento. A composição reduzida evita

que a suíte se torne lenta e frágil e mantém o E2E como confirmação de fluxos completos, e não como substituto dos testes unitários e de integração.

Da arquitetura do produto à arquitetura de validação

A divisão atual entre o monólito modular, o serviço de IA e as interfaces cliente determina uma sequência de validação coerente com a arquitetura existente. Uma alteração restrita a uma regra de negócio de um módulo do core deve ser verificada primeiro em nível unitário. Quando a alteração depende de PostgreSQL ou de uma API, a validação deve alcançar a integração correspondente. Quando o comportamento envolve análise realizada pelo serviço de IA, a integração deve verificar o contrato entre o core e a IA e, quando aplicável, o fluxo de mensagens no RabbitMQ. Somente os fluxos que exigem uma confirmação completa pela perspectiva do usuário devem chegar à camada E2E, utilizando as interfaces web e mobile conforme o fluxo protegido.

O ambiente de integração será sustentado por Docker Compose, com o monólito do core, o serviço de IA e as dependências necessárias executados de forma controlada. Para os cenários que utilizam PostgreSQL, a persistência deverá estar disponível antes do início da validação. Para os cenários de interação com IA, o serviço correspondente deverá estar operacional e o RabbitMQ deverá disponibilizar as filas e configurações necessárias. Os contratos de API entre as interfaces e o core, e entre o core e o serviço de IA quando aplicável, deverão estar definidos para que o teste de integração valide interfaces conhecidas, e não comportamentos ambíguos.

Na mensageria, o critério de sucesso envolverá a publicação e o consumo corretos das mensagens utilizadas na interação entre o core e o serviço de IA, além do comportamento de erro. A especificação da atividade determina que a fase de integração e mensageria apresente evidências do tratamento de falhas e do redirecionamento para Dead Letter Queue quando aplicável. Esse comportamento será previsto na arquitetura de teste para o fluxo de IA desde o início, mesmo que a execução aprofundada de resiliência pertença à etapa posterior do projeto.

Fluxo mínimo de validação automatizada



A sequência de validação parte de um commit ou Pull Request, verifica a compilação e a suíte unitária, avança para integração e mensageria quando a mudança exigir essas dependências e executa E2E nos

fluxos críticos selecionados. O gate de integração contínua representa o ponto em que os critérios definidos para aquela alteração precisam estar satisfeitos para que a mudança possa avançar.

Ferramentas e infraestrutura de teste

A seleção de ferramentas seguirá o papel de cada camada. Para os testes unitários, Pytest e Jest serão utilizados conforme a linguagem do componente que estiver sendo validado, mantendo a associação entre ferramenta e tecnologia do serviço. Para a interface e os testes de sistema, Playwright será utilizado para os fluxos E2E definidos como críticos. Para carga e desempenho, K6 será adotado na fase em que os endpoints críticos e os parâmetros de estresse estiverem suficientemente definidos.

A infraestrutura de integração utilizará Docker e Docker Compose, elementos já previstos na arquitetura do projeto. PostgreSQL fornecerá a persistência do core para os cenários que a exigirem, enquanto o RabbitMQ sustentará exclusivamente os fluxos de mensageria utilizados na comunicação com o serviço de IA. O frontend web e o aplicativo mobile serão considerados nas validações de sistema e E2E, de acordo com os fluxos críticos definidos. A automação será executada por GitHub Actions, que passará a concentrar as validações repetíveis do ciclo de código. Os resultados dos testes, logs e indicadores produzidos por essas execuções deverão permanecer como evidência da validação realizada.

A escolha das ferramentas não altera o princípio central do plano: cada camada deve existir porque responde a um risco específico. A ferramenta unitária acelera o feedback sobre regras locais; a ferramenta de integração demonstra comportamento entre componentes; a automação E2E confirma as jornadas críticas; e a ferramenta de carga prepara a medição de desempenho para a fase em que esse tipo de execução estiver previsto.

Integração contínua como gate de qualidade

O GitHub Actions será configurado para disparar a validação automática em commits e Pull Requests. O pipeline inicial deverá verificar a compilação e executar a suíte unitária do core e dos componentes que possuírem testes unitários automatizados. À medida que o ambiente de integração estiver pronto, serão incorporadas as validações de PostgreSQL e, nos fluxos que utilizarem IA, do serviço correspondente e do RabbitMQ. Os fluxos E2E classificados como críticos integrarão o pipeline antes do gate final correspondente, abrangendo as interfaces web e mobile conforme a funcionalidade protegida.

O fluxo de CI não substituirá a revisão de código, mas fornecerá evidências objetivas para essa revisão. Uma falha em uma etapa obrigatória deverá impedir o avanço automático da alteração até que a causa seja compreendida e corrigida ou até que exista uma decisão formal sobre o risco. Com isso, a integração contínua deixa de ser apenas uma etapa técnica de compilação e passa a representar uma barreira de qualidade associada ao fluxo de entrega.

A evolução do pipeline deverá ocorrer sem introduzir dependências desnecessárias. Primeiro será consolidada a validação unitária do core; depois serão acrescentadas as integrações que exigirem PostgreSQL e, de forma específica, as integrações entre o core, o RabbitMQ e o serviço de IA; em seguida, os E2E críticos das interfaces web e mobile; e, posteriormente, as etapas de desempenho e

resiliência previstas para a segunda entrega. A ordem reduz a complexidade inicial e permite que cada estágio seja estabilizado antes de receber o próximo.

Condições para iniciar e encerrar cada camada

A execução de testes unitários poderá começar quando o código da funcionalidade estiver commitado na branch de desenvolvimento e o pipeline base estiver configurado e compilando. A fase será considerada encerrada quando 100% dos testes unitários previstos estiverem passando e a cobertura de código atingir a meta mínima de 80% para o código relevante. A cobertura será tratada como indicador de alcance da suíte, não como substituto para a qualidade e a adequação dos casos de teste.

A integração e a mensageria poderão começar quando o ambiente estiver inicializado por Docker Compose, com o core, PostgreSQL, o serviço de IA e RabbitMQ operacionais conforme o cenário, e quando os contratos de API e de comunicação necessários estiverem definidos. O encerramento da fase exigirá a validação dos cenários aplicáveis de persistência e dos fluxos de publicação e consumo entre o core e a IA, acompanhada de evidências do tratamento de falhas e do redirecionamento para Dead Letter Queue quando o fluxo previsto utilizar esse mecanismo.

O planejamento de carga e desempenho terá como condição de entrada o mapeamento dos endpoints críticos e a definição da volumetria e dos parâmetros de estresse que serão utilizados. Como condição de saída desta primeira entrega, deverão existir cenários de teste de carga estruturados e prontos para execução posterior. Não serão atribuídos números de desempenho nesta fase, porque a atividade separa explicitamente o planejamento da execução final.

Essas condições estabelecem uma fronteira operacional clara. Uma camada não é considerada concluída apenas porque algum teste foi executado; é necessário atender às condições que demonstram que o nível de validação correspondente foi efetivamente coberto. Essa lógica permite que o avanço entre camadas seja verificável e evita declarar maturidade antes que existam evidências suficientes.

Métricas para acompanhar a evolução

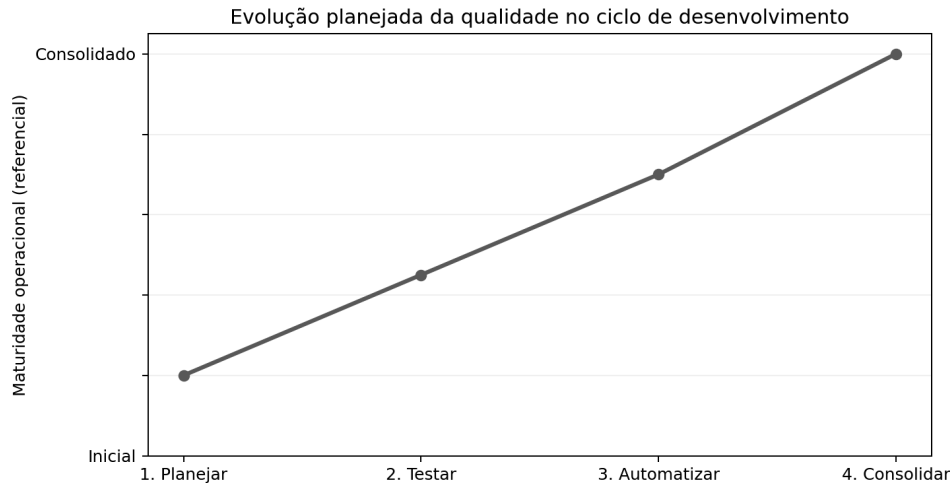
A adoção das validações será acompanhada por indicadores que permitam observar a estabilidade da suíte e a capacidade de fornecer feedback. A cobertura de código terá como referência inicial 80% na camada unitária. A taxa de sucesso dos testes mostrará a proporção de validações aprovadas por execução de pipeline, enquanto a quantidade de falhas por camada permitirá distinguir problemas de regra local, integração e mensageria ou E2E.

O tempo de execução do pipeline também será acompanhado porque a utilidade da automação depende de retorno suficientemente rápido para apoiar o ciclo de desenvolvimento. A evolução desses indicadores deverá ser observada em conjunto: aumentar a cobertura sem preservar a estabilidade, elevar a quantidade de E2E sem controlar o tempo de execução ou adicionar integrações sem garantir um ambiente reproduzível pode reduzir o valor do processo em vez de ampliá-lo.

Os indicadores de carga, resiliência, mutação e regressão serão incorporados quando as respectivas etapas de execução previstas na segunda entrega forem alcançadas. Até esse momento, permanecem como objetivos de medição e não como resultados do sistema.

Evolução planejada da qualidade

A implantação será progressiva. O primeiro estágio consolida o escopo, a pirâmide e os critérios de entrada e saída. O estágio seguinte estabelece a suíte unitária no core e o primeiro gate de CI. Depois, a validação se amplia para PostgreSQL e para os fluxos específicos que conectam o core ao serviço de IA por meio do RabbitMQ. Com essa base estabilizada, os fluxos críticos passam a ser protegidos por testes E2E nas interfaces web e mobile. Por fim, a segunda entrega amplia a cobertura para carga, escalabilidade, falhas de mensageria, DLQ, monitoramento, regressão e mutação, conforme o planejamento definido pelo Projeto Integrador.



A progressão preserva a relação entre custo e benefício da arquitetura atual. O teste unitário fornece a base mais rápida e barata para feedback sobre os módulos do core; a integração verifica as dependências reais com PostgreSQL e as interações específicas com o serviço de IA; o E2E confirma os fluxos críticos completos pelas interfaces web e mobile; e as etapas posteriores adicionam pressão, falha e análise mais profunda sobre um conjunto de validações já estabilizado. Dessa forma, cada novo nível reutiliza a base construída no anterior sem transformar a mensageria de IA em uma dependência obrigatória para todas as funcionalidades do sistema.

Consolidação do padrão de qualidade

A partir deste planejamento, uma alteração relevante deverá ser acompanhada por uma estratégia de validação proporcional ao seu risco. Regras de negócio e módulos do core deverão possuir testes unitários; comportamentos que atravessem persistência, APIs ou a integração específica com o serviço de IA deverão possuir validação de integração; os fluxos que utilizarem RabbitMQ deverão possuir

validações próprias de publicação, consumo e falha; fluxos críticos de negócio deverão ser cobertos por E2E nas interfaces web e mobile; e alterações que dependam de desempenho ou resiliência deverão ser direcionadas às etapas específicas previstas no ciclo posterior.

A qualidade será considerada parte do fluxo normal de engenharia quando os testes puderem ser executados de forma repetível, os resultados forem incorporados ao pipeline, os critérios de entrada e saída forem aplicados sem ambiguidade e as falhas encontradas retornarem para a suíte na forma de novas validações ou ajustes nos cenários existentes. O planejamento, portanto, estabelece uma sequência em que especificação, implementação, execução e evidência permanecem conectadas ao longo do desenvolvimento.